

Appunti di Calcolatori - Introduzione

Trascrizione dalle fonti del corso

Indice

1	Introduzione ai Calcolatori	3
1.1	Descrizione del Corso	3
1.2	Struttura e Modalità d'Esame	3
1.3	Evoluzione Storica: dall'ENIAC ad oggi	3
1.4	Perché studiare l'architettura?	3
1.5	Software di Sistema e Linguaggi	3
1.6	Componenti del Calcolatore	4
2	Aritmetica dei Calcolatori	4
2.1	Informazione nei Computer e Codifica	4
2.2	Basi di Numerazione e Conversioni	4
2.3	Aritmetica Binaria	5
2.4	Codifica dei Numeri Interi (con Segno)	5
2.5	Rappresentazione dei Numeri Reali	6
3	Esercizi di Aritmetica e Rappresentazione	7
4	Cenni alle Reti Logiche	17
4.1	Valori Logici ed Elettronica	17
4.2	Tipologie di Reti Logiche	17
4.3	Algebra di Boole e Porte Logiche	17
4.4	Circuiti Combinatori Fondamentali	18
4.5	Implementazioni Fisiche e Minimizzazione	18
4.6	Elementi di Memoria e Reti Sequenziali	18
5	Il linguaggio Assembly	19
5.1	Linguaggio macchina ed Assembly	19
5.2	Programmare in Assembly	20
5.3	Istruzioni Assembly e ISA	20
5.4	Funzionamento di una CPU e Programmi Assembly	20
5.5	Registri	21
5.6	Tipi di Istruzioni Assembly	21
5.7	Esecuzione condizionale e Predicati	22
5.8	Operandi, Destinazioni e Architetture (RISC vs CISC)	22
5.8.1	RISC (Reduced Instruction Set Computer)	22
5.8.2	CISC (Complex Instruction Set Computer)	22
5.9	Accessi alla memoria e Indirizzamento	23
5.10	ISA, ABI e Convenzioni di chiamata	23

6	Elenco Operandi e Istruzioni RISC-V	23
6.1	Operandi RISC-V	25
6.2	Istruzioni Aritmetiche e di Trasferimento Dati	25
6.3	Linguaggio Assembler: Istruzioni Logiche, Scorrimento e Salti	25
6.4	Dettaglio sulle Operazioni Logiche e di Scorrimento	25
6.5	Istruzioni per il Controllo del Flusso (Decisioni)	26
7	Gestione delle Procedure e della Memoria in RISC-V	27
7.1	Procedure e Protocollo	27
7.2	Protocollo di chiamata RISC-V	27
7.2.1	Le Istruzioni di Salto	27
7.3	Uso dello Stack e Chiamate a Procedura in C	28
7.3.1	Esempio e Fasi di Compilazione	28
7.3.2	Un bagno di realtà	29
7.4	Procedure Ricorsive e Ottimizzazioni	29
7.4.1	Esempio Pratico: La funzione Fattoriale	29
7.4.2	Tail Recursion (Ricorsione di coda)	30
7.5	Memoria e Variabili	30
7.5.1	Classi di Memoria e Record di Attivazione	30
7.5.2	Schema Generale di Memoria RISC-V	30
7.5.3	Riassunto Convenzioni Registri RISC-V	31
8	Esempi di Programmi Assembly RISC-V	31
8.1	Informazioni Generali e Scopo della Lezione	31
8.2	Semplici Istruzioni Aritmetiche e Logiche	31
8.2.1	Traduzione Base	31
8.2.2	Traduzione Ottimizzata (v2)	31
8.3	Accesso alla Memoria	32
8.4	Blocchi Condizionali e Cicli	32
8.4.1	Condizione di Uguaglianza (<code>if (i == j)</code>)	32
8.4.2	Condizione di Disuguaglianza (<code>if (i < j)</code>)	32
8.4.3	Ciclo While (<code>while (a[i] == k)</code>)	32
8.5	Funzioni Foglia e Non Foglia	32
8.5.1	Funzioni Foglia	32
8.5.2	Tabella dei Registri RISC-V	32
8.5.3	Funzioni Non Foglia	33
8.6	Algoritmi Complessi in Assembly	33
8.6.1	Ordinamento di Array (Insertion Sort)	33
8.6.2	Copia di Stringhe	33
9	Tutorato Assembly RISC-V	33
9.1	Concetti Teorici Fondamentali	33
9.2	Traduzione di Costrutti: dal C all'Assembly	33
10	Soluzioni Mockup Complete	35

1 Introduzione ai Calcolatori

1.1 Descrizione del Corso

Il corso, noto anche come **Architettura degli elaboratori**, si concentra principalmente su lezioni teoriche integrate da esercitazioni pratiche riguardanti l'aritmetica dei calcolatori e il linguaggio **Assembly**. In totale, l'insegnamento prevede **48 ore** di didattica frontale distribuite in 12 settimane.

1.2 Struttura e Modalità d'Esame

Il programma è suddiviso in tre parti principali:

1. **Prima parte:** Introduzione, aritmetica dei calcolatori e cenni sulle reti logiche.
2. **Seconda parte:** Linguaggio Assembly, con focus su architetture RISC-V, INTEL e ARM.
3. **Terza parte:** Elementi di architettura dei calcolatori, inclusi CPU, Pipeline e gerarchie di memoria.

L'esame prevede una **prova scritta**, spesso svolta su piattaforma Moodle con quesiti a risposta multipla, e un **orale opzionale**. Quest'ultimo è obbligatorio per voti compresi tra 15 e 17 o per confermare voti alti come la lode. Il libro di riferimento è *“Struttura e progetto dei calcolatori - Progettare con RISC-V”* di Patterson e Hennessy.

1.3 Evoluzione Storica: dall'ENIAC ad oggi

I calcolatori elettronici rappresentano oggi una tecnologia vitale che produce il 10% del PIL degli Stati Uniti.

- **ENIAC (1946):** Considerato uno dei primi calcolatori, fu commissionato per calcolare le traiettorie dei proiettili; occupava una stanza di 9x30 metri e consumava enormi quantità di energia.
- **La rivoluzione:** Attualmente i calcolatori pervadono ogni aspetto della vita, dalle automobili ai telefoni cellulari, fino alla robotica e alla mappatura del genoma.

1.4 Perché studiare l'architettura?

È essenziale per un programmatore conoscere l'organizzazione del calcolatore per scrivere software efficiente, comprendendo come sfruttare la **gerarchia di memoria** e il **parallelismo**. Le prestazioni dipendono dall'interazione tra algoritmi, compilatore, hardware e l'**ISA** (Instruction Set Architecture), che definisce le istruzioni riconosciute dalla CPU.

1.5 Software di Sistema e Linguaggi

Il software di sistema agisce come intermediario tra l'utente e l'hardware:

- **Sistema Operativo (SO):** Gestisce l'input/output, la memoria e il multitasking.
- **Compilatore:** Traduce il codice ad alto livello (come il C) in linguaggio macchina.

L'**Assembly** introduce codici mnemonici per rendere le istruzioni macchina gestibili dall'uomo prima della traduzione finale in bit.

1.6 Componenti del Calcolatore

Un calcolatore è composto da quattro elementi principali:

1. **Input:** Dispositivi per inviare dati, come tastiera e mouse.
2. **Output:** Dispositivi per visualizzare i risultati, come schermi e stampanti.
3. **Processore:** Esegue le elaborazioni, diviso in **datapath** (parte operativa) e **unità di controllo**.
4. **Memoria:** Unità dove vengono memorizzati dati e programmi in esecuzione.

2 Aritmetica dei Calcolatori

- **Corso:** CALCOLATORI - Aritmetica dei Calcolatori.
- **Docente:** Giovanni Iacca (basato su materiale del Prof. Luca Abeni).
- **Istituto:** Università degli Studi di Trento, Dipartimento di Ingegneria e Scienza dell'Informazione.

2.1 Informazione nei Computer e Codifica

- Un computer è costituito da circuiti elettronici in cui la corrente può passare (valore 1) o non passare (valore 0).
- Tutti i tipi di informazione (numeri, testo, programmi, immagini, suoni) sono memorizzati e manipolati come sequenze di 0 e 1.
- Una singola cifra binaria è detta **bit**, mentre sequenze di 8 bit formano un **byte**.
- La **codifica** è la funzione che associa a un oggetto o simbolo una specifica sequenza di bit.
- Una codifica su n bit permette di rappresentare 2^n entità o simboli distinti.

2.2 Basi di Numerazione e Conversioni

- In un sistema in base B , si usano B cifre (da 0 a $B - 1$).
- Il valore di un numero in un sistema posizionale si ottiene moltiplicando ogni cifra per una potenza della base dipendente dalla sua posizione: $\sum_{k=0}^i c_k B^k$.
- **Basi comuni:** Base 2 (binario), Base 8 (ottale), e Base 16 (esadecimale, che usa le lettere da A ad F per i valori da 10 a 15).
- Un numero binario su k cifre permette di esprimere valori da 0 a $2^k - 1$.
- **Conversioni:**
 - Da binario a esadecimale: ogni cifra esadecimale corrisponde esattamente a 4 cifre binarie.

- Da Base B a Base 10: si moltiplica ogni cifra per B^i in base alla sua posizione.
- Da Base 10 a Base B : si procede tramite divisione intera iterativa per B , dove il resto diventa la cifra da inserire e il quoziente viene riassegnato per l'iterazione successiva.

2.3 Aritmetica Binaria

- **Somma:** Si procede da destra a sinistra calcolando i riporti (Carry), in modo simile alla base 10.
- **Sottrazione:** È un'operazione con "prestito". Se necessario, si scorre a sinistra fino a trovare un 1, invertendo i bit incontrati (es. $10..0 \rightarrow 01..1$).
- **Moltiplicazione per potenze di 2:** Aggiungere uno 0 a destra in base 2 equivale a moltiplicare per 2 (*shifting*), mentre eliminare la cifra più a destra equivale a dividere per 2.
- La moltiplicazione di due numeri naturali si esegue con l'applicazione di shifting multipli e somme.

2.4 Codifica dei Numeri Interi (con Segno)

Dovendo rappresentare numeri interi $z \in \mathbb{Z}$, si utilizzano diverse tecniche per indicare i numeri negativi:

1. **Modulo e Segno:** Si usa 1 bit per il segno (0 per positivo, 1 per negativo) e $k - 1$ bit per il valore assoluto. Questo porta al problema di avere due diverse codifiche per lo zero ($+0$ e -0).
2. **Complemento a 1:** I numeri positivi sono rappresentati in modulo; i negativi sono il complemento a 1 (inversione dei bit) del valore assoluto. Anch'esso presenta due rappresentazioni per lo zero.
3. **Complemento a 2:** Per calcolare un numero negativo, si fa il complemento a 1 e si aggiunge 1.
 - In questo sistema la codifica dello zero è unica.
 - Permette di codificare valori da -2^{k-1} a $2^{k-1} - 1$.
 - Rende l'operazione di somma molto più semplice.

Overflow nell'Aritmetica Intera

- L'overflow si verifica quando il risultato di un'operazione non è rappresentabile sui k bit disponibili.
- Avviene solo se si sommano o sottraggono due numeri con lo stesso segno.
- Più precisamente, c'è overflow se la somma $a + b > 2^{k-1} - 1$ (overflow positivo) oppure se $a + b < -2^{k-1}$ (overflow negativo).
- Questi comportamenti si basano sull'aritmetica modulare (classi di equivalenza modulo 2^k), simile ai numeri sul quadrante di un orologio.

2.5 Rappresentazione dei Numeri Reali

Non essendo sempre possibile rappresentare un numero reale con un numero finito di bit (es. numeri irrazionali), si utilizzano delle approssimazioni.

Virgola Fissa (Fixed Point)

- Si dedicano $k - f$ cifre alla parte intera e f cifre alla parte frazionaria.
- Vantaggio: permette di usare operazioni matematiche semplici come per i numeri interi, riscaldando i risultati, semplificando così l'architettura della CPU.
- Svantaggio: diventa difficile trattare nella stessa applicazione numeri molto grandi e numeri molto piccoli.

Virgola Mobile (Floating Point - IEEE 754)

- Un numero reale si esprime come $x = M \cdot B^E$ (in informatica la base $B = 2$), dove M è la mantissa e E l'esponente.
- Lo standard IEEE 754 utilizza una codifica normalizzata in C per esponenti traslati $E' = E + 2^{e-1} - 1$.
- **Numeri Normalizzati:** Si usano quando $E' > 0$. La mantissa viene rappresentata con la forma $1.M$ e l'esponente vale $E = E' - (2^{e-1} - 1)$.
- **Numeri Denormalizzati:** Si usano quando $E' = 0$, per gestire l'underflow vicini allo zero. La mantissa assume forma $0.M$.
- **Casi Particolari:**
 - **Zero:** Esponente 0 e mantissa 0.
 - **Infinito:** Esponente tutto a 1 (es. 255 per float 32-bit) e mantissa 0.
 - **NaN (Not a Number):** Esponente tutto a 1 e mantissa diversa da zero.

Tipi di Floating Point in C

- **float (Precisione Singola):** Usa 32 bit (8 per l'esponente, 23 per la mantissa).
- **double (Precisione Doppia):** Usa 64 bit (11 per l'esponente, 52 per la mantissa).
- **long double:** Può usare 80 o 128 bit.

Per i `float` a singola precisione, il massimo valore normalizzato rappresentabile è di circa $3.4 \cdot 10^{38}$, mentre il minimo numero positivo denormalizzato rappresentabile è circa $1.4 \cdot 10^{-45}$.

3 Esercizi di Aritmetica e Rappresentazione

Domanda 30: Decimale in Binario

Indicare l'esatto corrispondente in binario di 728_{10}

- ☐ Nessuna delle altre risposte
- ☐ 000001101110_2
- ☒ 001011011000_2
- ☐ 000001101101_2
- ☐ 000111011000_2

Svolgimento:

728		0
364		0
182		0
91		1
45		1
22		0
11		1
5		1
2		0
1		1

Risposta: $728_{10} = 001011011000_2 \rightarrow$ Opzione #3.

Domanda 30: Binario in Decimale

Come si rappresenta in decimale il numero binario 00100110_2 ?

- ☐ 76_{10}
- ☐ Nessuna delle altre risposte
- ☒ 38_{10}
- ☐ 100_{10}
- ☐ 25_{10}

Svolgimento:

$$\begin{aligned} 00100110_2 &= 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^5 \\ &= 2 + 4 + 32 \\ &= 38 \end{aligned}$$

Risposta: $= 38_{10} \rightarrow$ Opzione #3.

Domanda 30: Decimale in Binario

Indicare l'esatto corrispondente in binario di 3429_{10}

- ☐ Nessuna delle altre risposte
- ☐ 101101101010_2
- ☒ 110101100101_2
- ☐ 101001101011_2
- ☐ 010101101101_2

Svolgimento:

3429	1
1714	0
857	1
428	0
214	0
107	1
53	1
26	0
13	1
6	0
3	1
1	1

Risposta: $3429_{10} = 110101100101_2 \rightarrow$ Opzione #3.

Domanda 30: Binario in Decimale

Come si rappresenta in decimale il numero binario 101101101100_2 ?

- ☐ Nessuna delle altre risposte
- ☐ 5848_{10}
- ☒ 2924_{10}
- ☐ 877_{10}
- ☐ 731_{10}

Svolgimento:

$$\begin{aligned} 101101101100_2 &= 1 \times 2^2 + 1 \times 2^3 + 1 \times 2^5 + 1 \times 2^6 + 1 \times 2^8 + 1 \times 2^9 + 1 \times 2^{11} \\ &= 4 + 8 + 32 + 64 + 256 + 512 + 2048 \\ &= 2924 \end{aligned}$$

Risposta: $101101101100_2 = 2924_{10} \rightarrow$ Opzione #3.

Domanda 1: Somma Binaria

Usando la rappresentazione binaria, svolgere la somma $623 + 412$

- ☐ $623_{10} + 412_{10} = 00111111011_2$
- ☐ $623_{10} + 412_{10} = 10111110011_2$
- ☐ Nessuna delle altre risposte
- ☐ $623_{10} + 412_{10} = 101100000100_2$
- ☒ $623_{10} + 412_{10} = 01000001011_2$

Svolgimento conversioni:

623	1	412	0
311	1	206	0
155	1	103	1
77	1	51	1
38	0	25	1
19	1	12	0
9	1	6	0
4	0	3	1
2	0	1	1
1	1		

Risposta: $(623 + 412)_{10} = 01000001011_2 \rightarrow$ Opzione #5.

Domanda 1: Somma Binaria

Usando la rappresentazione binaria, svolgere la somma $8494 + 7726$

- ☐ $8494_{10} + 7726_{10} = 1100001111110101_2$
- ☐ $8494_{10} + 7726_{10} = 1111010111000011_2$
- ☐ Nessuna delle altre risposte
- ☐ $8494_{10} + 7726_{10} = 010111000011111_2$
- ☒ $8494_{10} + 7726_{10} = 001111101011100_2$

Risposta: $(8494 + 7726)_{10} = 001111101011100_2 \rightarrow$ Opzione #5.

Domanda 1: Sottrazione Binaria

Usando la rappresentazione binaria, svolgere la sottrazione:

$$\begin{array}{r} 101100111011000110010100 \\ - 010101010100101010100101 \\ \hline \end{array}$$

- ☐ 101100010000100001000_2
- ☐ 101110011000000001000_2
- ☐ Nessuna delle altre risposte
- ☐ 01100001000010010100_2
- ☒ 01101001000010000100_2

Risposta: $01101001000010000100_2 \rightarrow$ Opzione #5.

Domanda 21: Moltiplicazione Binaria

Svolgere in binario la moltiplicazione: 21×11

- ☐ Nessuna delle altre risposte
- ☒ 11100111_2
- ☐ 10010111_2
- ☐ 10011010_2
- ☐ 11110011_2

Svolgimento conversioni:

21		1	11		1
10		0	5		1
5		1	2		0
2		0	1		1
1		1			

Risposta: $(21 \times 11)_{10} = 11100111_2 \rightarrow$ Opzione #2.

Domanda 21: Moltiplicazione Binaria

Svolgere in binario la moltiplicazione: 85×19

☐ Nessuna delle altre risposte

☒ 011001001111_2

☐ 100001001101_2

☐ 011100101101_2

☐ 010111001111_2

Risposta: $(85 \times 19)_{10} = 011001001111_2 \rightarrow$ Opzione #2.

Domanda 5: Complemento a 2

Come è rappresentato -97 in complemento a 2 su 8 bit?

☐ 01111010_2

☒ 10011111_2

☐ 01100001_2

☐ Nessuna delle altre risposte

☐ 11100001_2

Svolgimento:

97		1
48		0
24		0
12		0
6		0
3		1
1		1

$97_{10} = 01100001_2$

Invertendo i bit e sommando 1 otteniamo il complemento a 2.

Risposta: $CA2(-97) = 10011111_2 \rightarrow$ Opzione #2.

Domanda 5: Operazione in Complemento a 2 (12 bit)

Svolgere in complemento a 2 su 12 bit l'operazione $1957 - 2016$

- ☐ 111110000101_2
- ☒ 111111000101_2
- ☐ 111101011100_2
- ☐ Nessuna delle altre risposte
- ☐ 000000111011_2

Risposta: $(1957 - 2016)_{10} = 111111000101_2 \rightarrow$ Opzione #2.

Domanda 5: Operazione in Complemento a 2 (8 bit)

Svolgere in complemento a 2 su 8 bit l'operazione $-34 - 53$

- ☐ 11101101_2
- ☒ 10101001_2
- ☐ 00010011_2
- ☐ Nessuna delle altre risposte
- ☐ 01010111_2

Svolgimento conversioni:

34		0	53		1
17		1	26		0
8		0	13		1
4		0	6		0
2		0	3		1
1		1	1		1

$$34_{10} = 00100010_2$$

$$53_{10} = 00110101_2$$

Risposta: $(-34 - 53)_{10} = 10101001_2 \rightarrow$ Opzione #2.

Domanda 58: Virgola Fissa a Decimale

Convertire in decimale il binario a virgola fissa 1001110.0011_2

- ☐ $1001110.0011_2 = 156.1875_{10}$
- ☐ $1001110.0011_2 = 78.375_{10}$
- ☐ $1001110.0011_2 = 156.375_{10}$

■ $1001110.0011_2 = 78.1875_{10}$

☐ Nessuna delle altre risposte

Svolgimento:

$$\begin{aligned} 1001110.0011_2 &= 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 + 1 \times 2^6 + 1 \times 2^{-3} + 1 \times 2^{-4} \\ &= 2 + 4 + 8 + 64 + 0.125 + 0.0625 \\ &= 78.1875 \end{aligned}$$

Risposta: $1001110.0011_2 = 78.1875_{10} \rightarrow$ Opzione #4.

Domanda 58: Decimale a Virgola Fissa

Convertire in binario a virgola fissa il decimale 362.828125_{10}

☐ $362.828125_{10} = 010101101.110101_2$

☐ $362.828125_{10} = 010101101.101011_2$

☐ $362.828125_{10} = 101101010.101011_2$

■ $362.828125_{10} = 101101010.110101_2$

☐ Nessuna delle altre risposte

Svolgimento (Parte Intera e Frazionaria):

362	0	0.828125	1
181	1	0.65625	1
90	0	0.3125	0
45	1	0.625	1
22	0	0.25	0
11	1	0.5	1
5	1		
2	0		
1	1		

Risposta: $362.828125_{10} = 101101010.110101_2 \rightarrow$ Opzione #4.

Domanda 58: Somma in Virgola Fissa

Riportare in binario il risultato della somma $10.5625_{10} + 1100.1011_2$

☐ $10.5625_{10} + 1100.1011_2 = 10110.01_2$

☐ $10.5625_{10} + 1100.1011_2 = 11111.01_2$

☐ $10.5625_{10} + 1100.1011_2 = 10010.01_2$

☒ $10.5625_{10} + 1100.1011_2 = 10111.01_2$

☐ Nessuna delle altre risposte

Risposta: $10.5625_{10} + 1100.1011_2 = 10111.01_2 \rightarrow$ Opzione #4.

Domanda 76: Conversione IEEE754

Rappresentare il decimale -1313.3125 secondo lo standard IEEE754.

☐ Nessuna delle altre risposte

☐ 0100 0100 1010 0100 0010 1010 0000 0000

☐ 1100 0100 1101 0010 0001 0101 0000 0000

☒ 1100 0100 1010 0100 0010 1010 0000 0000

☐ 0100 0100 1101 0010 0001 0101 0000 0000

Svolgimento:

1. **Rappresentazione binaria:** 10100100001.0101_2

2. **Scorrere la virgola:** $10100100001.0101 \rightarrow 1.01001000010101 \times 2^{10}$ ($p = 10$)

3. **Bit di segno:** Numero negativo $\rightarrow 1$

4. **Esponente:**

$$\begin{aligned} E &= bias + p \\ &= 127_{10} + 10_{10} \\ &= 137_{10} \rightarrow 10001001_2 \end{aligned}$$

5. **Mantissa:** 010010000101010000000000

Risposta: $S = 1 \mid e = 10001001 \mid m = 010010000101010000000000$
 $\rightarrow$ Opzione #4.

Domanda 76: Da IEEE754 Esadecimale a Decimale

A quale numero decimale corrisponde la cifra esadecimale $0xE7E80000$ codificata secondo lo standard IEEE754?

☐ Nessuna delle altre risposte

☐ Corrisponde al decimale -1.5625000×2^{16}

☐ Corrisponde al decimale -1.5546875×2^{18}

☒ Corrisponde al decimale -1.8125000×2^{80}

☐ Corrisponde al decimale -1.8046875×2^{82}

Svolgimento:

1. **Rappresentazione binaria:** 1110 0111 1110 1000 0000 0000 0000 0000

2. **Scomposizione:** $S = 1 \mid e = 11001111 \mid m = 110100000000000000000000$

3. **Bit di segno:** $1 \rightarrow$ Numero negativo.

4. **Esponente:**

$$\begin{aligned} 11001111_2 &= 127_{10} + p \\ 207_{10} &= 127_{10} + p \rightarrow p = 80_{10} \end{aligned}$$

5. **Mantissa:**

$$\begin{aligned} M &= 1.1101_2 = 1 + 1 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-4} \\ &= 1 + 0.5 + 0.25 + 0.0625 = 1.8125_{10} \end{aligned}$$

Risposta: $-1.8125 \times 2^{80} \rightarrow$ Opzione #4.

Domanda 76: Rappresentazioni Equivalenti

A quale delle seguenti opzioni corrisponde la cifra decimale 3.5_{10} ?

☐ Corrisponde al numero $1.11_2 \times 2^1$

☐ Corrisponde al numero 11.1_2

☐ Corrisponde al numero $111_2 \times 2^{-1}$

☒ Tutte le opzioni sono corrette

☐ Corrisponde al numero $0.111_2 \times 2^2$

Svolgimento:

$$\begin{aligned} 1.11_2 \times 2^1 &= (1 + 0.5 + 0.25) \times 2 = 1.75 \times 2 = 3.5_{10} \\ 11.1_2 &= 2 + 1 + 0.5 = 3.5_{10} \\ 111_2 \times 2^{-1} &= (4 + 2 + 1) \times 0.5 = 7 \times 0.5 = 3.5_{10} \\ 0.111_2 \times 2^2 &= (0.5 + 0.25 + 0.125) \times 4 = 0.875 \times 4 = 3.5_{10} \end{aligned}$$

Risposta: Tutte le opzioni sono equivalenti $\rightarrow$ Opzione #4.

Domanda 76: Da IEEE754 Binario a Decimale

A quale numero decimale corrisponde la seguente cifra binaria codificata secondo lo standard IEEE754?

$S = 1 \mid e = 10000101 \mid m = 111011010100000000000000$

- ☐ Nessuna delle altre risposte
- ☐ Corrisponde al decimale 118.625
- ☐ Corrisponde al decimale $-1.926757813 \times 2^{139}$
- ☒ Corrisponde al decimale -123.3125
- ☐ Corrisponde al decimale -118.625

Svolgimento:

1. **Segno:** $1 \rightarrow$ Negativo.
2. **Esponente:** $10000101_2 = 133_{10}$. Quindi l'esponente reale è $133 - 127 = 6$.
3. **Mantissa:** Scostando la virgola di 6 posizioni $\rightarrow 111011.0101_2$

$$\begin{aligned} 111011.0101_2 &= 32 + 16 + 8 + 2 + 1 + 0.25 + 0.0625 \\ &= 123.3125 \end{aligned}$$

Risposta: $-123.3125_{10} \rightarrow$ Opzione #4.

Domanda 76: Conversione IEEE754

Rappresentare il decimale 5462.875 secondo lo standard IEEE754.

- ☐ Nessuna delle altre risposte
- ☐ 0100 0101 1101 0101 0101 1011 1000 0000
- ☐ 0100 0101 0101 0101 0110 1110 0000 0000
- ☐ 0100 0101 1010 1010 1011 0111 0000 0000
- ☒ 0100 0101 0110 1010 1011 0111 0000 0000

Svolgimento:

1. **Rappresentazione binaria:** 1010101010110.111_2
2. **Scorrere la virgola:** $1.010101010110111 \times 2^{12}$ ($p = 12$)
3. **Bit di segno:** Numero positivo $\rightarrow 0$
4. **Esponente:**

$$\begin{aligned} E &= 127 + 12 = 139_{10} \\ &\rightarrow 10001011_2 \end{aligned}$$

5. **Mantissa:** 010101010110111000000000

Risposta: $S = 0 \mid e = 10001011 \mid m = 010101010110111000000000$
 $\rightarrow$ Opzione #4.

4 Cenni alle Reti Logiche

L'obiettivo principale di questo corso è mostrare come funziona, e come si progetta, un calcolatore. Per fare ciò, è necessaria una piccola digressione su come si progettano i circuiti logici.

4.1 Valori Logici ed Elettronica

I computer moderni sono realizzati tramite circuiti elettronici. Trattandosi di elementi digitali avremo due livelli fondamentali:

- **Alto, asserito (1)**: Associato alla tensione di alimentazione V_{dd} .
- **Basso, negato (0)**: Associato alla massa, ovvero a una tensione nulla.

Altri livelli di tensione sono considerati non significativi e vengono assunti solo in fase transitoria.

4.2 Tipologie di Reti Logiche

Le reti logiche sono circuiti che trasformano alcuni valori logici in ingresso in altri valori logici in uscita. Si dividono in due tipi principali:

- **Reti Combinatorie**: Rappresentano una relazione funzionale tra ingresso e uscita. Non hanno memoria, poiché l'uscita dipende solo dal valore attuale dell'ingresso. Possono essere specificate tramite una *tabella di verità*.
- **Reti Sequenziali**: L'uscita dipende dalla storia degli ingressi passati e non solo dal valore attuale. Esse possiedono una memoria, definita anche come *stato* della rete.

4.3 Algebra di Boole e Porte Logiche

Le funzioni logiche combinatorie possono essere specificate in maniera compatta tramite espressioni algebriche definite con l'algebra di Boole. Esistono tre operatori di base, implementati fisicamente da **porte logiche**:

- **AND ($\cdot$)**: Rappresentato tramite il simbolo di prodotto, produce 1 se entrambi gli operandi sono 1, e 0 negli altri casi.
- **OR ($+$)**: Rappresentato tramite il simbolo della somma, produce 0 se entrambi gli operandi sono 0, e 1 negli altri casi.
- **NOT ($\bar{A}$)**: Rappresentato da una barra (o da un cerchietto nei circuiti), inverte il valore logico dell'ingresso.

Esistono semplici regole che ci permettono di manipolare e semplificare le espressioni logiche:

- **Identità**: $A + 0 = A$, $A \cdot 1 = A$.
- **Regola zero e uno**: $A + 1 = 1$, $A \cdot 0 = 0$.

- **Regola dell'inversa:** $A + \overline{A} = 1$, $A \cdot \overline{A} = 0$.
- **Commutativa:** $A + B = B + A$, $A \cdot B = B \cdot A$.
- **Associativa:** $A + (B + C) = (A + B) + C$, $A \cdot (B \cdot C) = (A \cdot B) \cdot C$.
- **Distributiva:** $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$, $A + (B \cdot C) = (A + B) \cdot (A + C)$.

Di grandissima importanza sono le **leggi di De Morgan**:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Queste regole dimostrano che, avendo a disposizione un elemento logico NAND o NOR, è possibile ricavare tutti gli altri operatori.

4.4 Circuiti Combinatori Fondamentali

Tra i blocchi logici più comuni troviamo:

- **Decoder:** Un circuito che converte un codice binario in ingresso nelle singole uscite corrispondenti.
- **Multiplexer (Mux):** Un deviatore che, sulla base di un input di controllo, determina quale degli input debba passare in uscita.

Molto spesso si costruiscono array di questi elementi logici per operare su dati complessi (ad esempio multiplexer a 32 bit), trattando l'insieme di fili (BUS) come un singolo segnale logico.

4.5 Implementazioni Fisiche e Minimizzazione

L'espressione logica equivalente a una tabella di verità si può ricavare con la **forma canonica SP**: si prende ciascuna riga uguale a 1 e si scrive un termine di prodotto logico (AND), per poi fare la somma (OR) di tutti i prodotti individuati.

- **PLA (Programmable Logic Array):** Struttura logica hardware composta da due stadi: una barriera di AND (mintermini) e una barriera di OR.
- **Costo:** Il *costo* di una rete logica è definito come la somma del numero di porte e del numero di ingressi della rete.
- **Sintesi e Minimizzazione:** Esistono metodi sistematici basati sull'applicazione iterativa delle regole algebriche, o grafici (mappe di Karnaugh), che consentono di semplificare l'espressione in modo "furbo" e ridurre il costo computazionale.

4.6 Elementi di Memoria e Reti Sequenziali

In molte applicazioni è necessario introdurre una "memoria", la quale si ottiene tramite **reazione** (o retroazione): l'uscita di alcune porte viene reindirizzata in ingresso ad altre porte del medesimo stadio. Così facendo, gli ingressi "ricordano" il passato della rete.

- **R-S Latch:** Elemento bistabile fondamentale (Set-Reset) per memorizzare un bit. Ha il difetto critico di assumere uno stato indecidibile quando entrambi gli ingressi sono a 1.
- **Clock e Temporizzazione:** Dato che i segnali si propagano in tempo finito, gli elementi di memoria sono governati da un segnale speciale chiamato *clock* (gated latch). Se il clock è a zero, la rete è forzata e non può cambiare stato.
- **Latch D (Data):** Risolve il problema dello stato indecidibile forzando gli ingressi al circuito base tramite un'unica variabile e il suo negato, garantendo il funzionamento senza ambiguità durante la fase positiva del clock.
- **Flip-Flop Master-Slave:** Configurazioni più complesse che consentono all'uscita di commutare esattamente e unicamente al termine dell'impulso di clock (*edge-triggered*).
- **Registri:** Insiemi di latch edge-triggered impiegati per memorizzare *word* (parole di dati) complesse; caricano l'input sul fronte in salita del clock operando come barriera.
- **Macchine a Stati:** La metodologia edge-triggered previene situazioni di corsa indecidibili (*race condition*). Porta alla realizzazione di macchine a stati in cui lo stato successivo dipende da quello presente e dagli ingressi (es. automi di Mealy o Moore).

5 Il linguaggio Assembly

5.1 Linguaggio macchina ed Assembly

- **CPU:** "capisce" (e riesce ad eseguire) solo il suo linguaggio macchina, ovvero una sequenza di 0 e 1.
- Non è un linguaggio propriamente utilissimo per noi umani.
- **Programmatore:** scrive programmi in linguaggi di alto livello (es. C, C++, Java, ecc.).
- Questi linguaggi, però, non sono direttamente comprensibili dalla CPU.
- Come riempire il gap?
- Quali sono i compromessi fra le sequenze di 0 e 1 e i linguaggi ad alto livello?
- **Assembly!:** Utilizza codici mnemonici invece di sequenze di 0 e 1, risultando molto più facile da usare.
- Vi è una stretta corrispondenza fra istruzioni macchina ed Assembly, rimanendo sempre un linguaggio strettamente legato alla CPU.

5.2 Programmare in Assembly

- Un **programma Assembly** è un file ASCII contenente la descrizione testuale delle istruzioni.
- Deve essere compilato per essere eseguito dalla CPU.
- **Assembler**: è il compilatore che traduce da Assembly a linguaggio macchina.
- Ad ogni istruzione Assembly corrisponde un'istruzione in linguaggio macchina.
- Salvo rare eccezioni (macro, label, riordinamento istruzioni), le istruzioni Assembly dipendono dalla CPU. Di conseguenza, i programmi **non sono portabili**.
- Linguaggio intermedio o linguaggio di programmazione?

5.3 Istruzioni Assembly e ISA

- L'insieme delle istruzioni (sequenze di bit) riconosciute da una CPU prende il nome di **Instruction Set Architecture (ISA)**.
- Non si tratta di una semplice lista di istruzioni, in quanto possiede una propria **Sintassi e Semantica**.
- Regola l'accesso ai dati definendo i **Registri** (tipo e numero) e le modalità di accesso alla memoria (indirizzamento).
- L'ISA di una CPU ne definisce direttamente anche il linguaggio Assembly.
- In alcuni casi vi sono più ISA per una singola CPU, portando a diversi linguaggi Assembly (es. Intel 32-bit vs. Intel 64-bit).

5.4 Funzionamento di una CPU e Programmi Assembly

La struttura di un programma Assembly deriva direttamente dal meccanismo di funzionamento di una CPU:

1. **Fetch**: preleva l'istruzione dalla memoria. L'indirizzo è memorizzato in un apposito registro chiamato **Program Counter (PC)** o **Instruction Pointer (IP)**.
 2. **Decode**: decodifica l'istruzione per capire cosa fare.
 3. **Execute**: esegue l'istruzione (operazione aritmetico/logica, accesso alla memoria, ecc.).
- Un programma è sostanzialmente una lista di istruzioni macchina eseguite, prevalentemente, in ordine sequenziale.
 - Si tratta perlopiù di operazioni aritmetico/logiche o talvolta di movimento dati.
 - Ordine sequenziale: esistono operazioni di **controllo del flusso** (salti) per rompere l'esecuzione sequenziale modificando il valore di PC/IP.

- Essendo un linguaggio di basso livello, si opera con salti, non esistono costrutti diretti per cicli o selezioni.
- Le istruzioni operano su dati (operandi): possono essere operandi di operazioni aritmetiche, dati da muovere da/verso la memoria o nuovi valori per PC/IP.

5.5 Registri

Le istruzioni Assembly operano su dati che possono essere:

- **Immediati** (costanti).
- Contenuti in **registri**.
- Contenuti in memoria (sfruttando varie modalità di indirizzamento).
- Un **Registro a k bit** è fisicamente composto da una batteria di k flip-flop di tipo D. L'insieme forma un banco di registri utilizzabili dalle istruzioni Assembly.
- Cambiano da CPU a CPU e sono definiti dall'ISA.
- Si dividono in registri *general-purpose* e registri specializzati.
- In genere, il numero di registri varia da 4 a 64 e la sintassi cambia a seconda dell'Assembly (possono avere nomi simbolici o numeri).

5.6 Tipi di Istruzioni Assembly

Ogni istruzione Assembly è formata da un codice mnemonico seguito da eventuali operandi (immediati, in registri o in memoria), con eventuali vincoli imposti dall'ISA. Le istruzioni si dividono in:

- **Aritmetico / logiche**: implementate dalla ALU (Unità Logica Aritmetica). Specificano in genere due operandi ed una destinazione (talvolta implicita).
- **Movimento dati**: spostano dati fra registri, caricano costanti in registri/memoria o muovono dati tra memoria e registri.
- **Controllo del flusso (salti)**: manipolano il contenuto di PC/IP. Permettono l'esecuzione condizionale e la gestione di invocazioni/ritorni da subroutine (che richiedono il salvataggio preventivo del valore di PC/IP).
- **RISC**: semplifica l'implementazione della CPU.
- **CISC**: semplifica la scrittura di programmi Assembly.

5.7 Esecuzione condizionale e Predicati

- Alcune istruzioni vanno eseguite solo se si verificano determinate condizioni (necessario per implementare selezioni e cicli).
- In alcune ISA (come ARM), tutte le istruzioni possono avere esecuzione condizionale.
- Come esprimere queste condizioni?
 - Confronto fra valori di registri *general-purpose*. In alcune ISA si usa un'istruzione come *set if less than* per settare un terzo registro in base al risultato del confronto.
 - Valutazione di **flag** contenuti in uno speciale registro. I flag sono settati da istruzioni aritmetiche, logiche o di confronto (es. `cmp`, che esegue una sottrazione scartando il risultato ma aggiornando i flag) per indicare risultati pari a 0, negativi, overflow, ecc.

5.8 Operandi, Destinazioni e Architetture (RISC vs CISC)

Sulla gestione degli operandi e della destinazione, esistono due "filosofie" di design delle ISA:

1. **Tutto nei registri** (o operandi immediati): rende l'implementazione della CPU più semplice ed elegante. Porta ad architetture **RISC**.
2. **Possibilità di avere operandi o destinazione in memoria**: rende il linguaggio Assembly più potente e semplice da usare. Porta ad architetture **CISC**.

5.8.1 RISC (Reduced Instruction Set Computer)

- **Obiettivo**: semplificare al massimo la struttura della CPU. Offre un'ISA più "semplice" e regolare.
- Le istruzioni aritmetico/logiche operano solo sui registri (nessun operando in memoria): `<opcode> <dst>, <arg1>, <arg2>`.
- L'accesso alla memoria è delegato esclusivamente a due istruzioni: `load` e `store`.
- Conseguenze: servono più registri e la codifica delle istruzioni avviene su un numero fisso di bit.

5.8.2 CISC (Complex Instruction Set Computer)

- **Obiettivo**: fornire istruzioni Assembly più potenti, flessibili e performanti.
- Tutte le istruzioni possono avere operandi e/o destinazione in memoria (spesso con vincoli, es. in Intel al massimo un operando può essere in memoria).
- Maggior numero di istruzioni con comportamenti complessi, sintassi meno regolare e codifica su un numero variabile di bit.
- Esempio: per "salvare" alcuni bit di codifica, le istruzioni Intel hanno destinazione implicita.

Meglio CISC o RISC? Come in ogni cosa, gli estremismi non sono l'ideale: le soluzioni di maggior successo mescolano i due approcci. Intel è tipicamente CISC ma integra concetti RISC; ARM è una RISC "pragmatica" con alcune peculiarità avanzate.

5.9 Accessi alla memoria e Indirizzamento

Le istruzioni che accedono alla memoria (**load/store** per RISC, istruzioni generiche per CISC) richiedono l'argomento **<memory location>**. Le **modalità di indirizzamento** permettono di esprimere questa locazione in vari modi:

- **Assoluto**: indirizzo costante codificato direttamente nell'istruzione (risulta problematico in RISC per il numero fisso di bit della codifica).
- **Indiretto**: registro che contiene l'indirizzo codificato nell'istruzione (utile per implementare puntatori).
- **Base + Spiazzamento (2+1)**: indirizzo ottenuto sommando un registro ad un valore immediato.
- **Base + Indice (2+3)**: indirizzo ottenuto sommando un registro ad un registro scalato/shiftato (utile per implementare array).
- **Base + Indice + Spiazzamento (1+2+3)**: indirizzo ottenuto sommando un registro, un registro scalato e un valore immediato.

Tradizionalmente le ISA RISC offrono modalità di indirizzamento più semplici, mentre ISA CISC (e ARM) permettono logiche di shift e pre/post-incremento utili per iterare nativamente sugli array.

5.10 ISA, ABI e Convenzioni di chiamata

- L'**ISA** si limita a definire le istruzioni riconosciute, il numero e il tipo di registri. Non forza, a livello hardware, come usare tali registri.
- **ABI (Application Binary Interface)**: è un insieme di convenzioni software, fondamentale per far comunicare correttamente compilatori, librerie e Sistema Operativo.
- Le **convenzioni di chiamata** (specificate dall'ABI e non dall'ISA) definiscono ad esempio se passare i parametri tramite stack o registri, e quali registri debbano rimanere inalterati al ritorno da una subroutine.

6 Elenco Operandi e Istruzioni RISC-V

L'architettura RISC-V si basa su uno specifico set di operandi e istruzioni che permettono l'interazione tra processore e memoria, l'esecuzione di calcoli aritmetico-logici e il controllo del flusso del programma.

Nome	Esempio	Commenti
32 registri	x0-x31	Accesso veloce ai dati. Nel RISC-V gli operandi devono essere contenuti nei registri per potere eseguire delle operazioni. Il registro x0 contiene sempre il valore 0.
Memoria	Memoria[0], Memoria[8]...	Alla memoria si accede solamente attraverso istruzioni di trasferimento dati. Il RISC-V utilizza l'indirizzamento al byte, perciò due variabili ampie due parole (double word) hanno indirizzi in memoria a distanza 8. La memoria consente di memorizzare strutture dati, vettori, o il contenuto dei registri.

Tipo di istruzioni	Istruzioni (Esempio)	Significato	Commenti
Aritmetiche	add x5, x6, x7	$x5 = x6 + x7$	Operandi in tre registri.
Aritmetiche	sub x5, x6, x7	$x5 = x6 - x7$	Operandi in tre registri.
Aritmetiche	addi x5, x6, 20	$x5 = x6 + 20$	Utilizzata per sommare delle costanti.
Trasferimento dati	ld x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di una parola doppia da memoria a registro.
Trasferimento dati	sd x5, 40(x6)	$\text{Memoria}[x6 + 40] = x5$	Spostamento di una parola doppia da registro a memoria.
Trasferimento dati	lw x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di una parola da memoria a registro.
Trasferimento dati	lwu x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di una parola senza segno da memoria a registro.
Trasferimento dati	sw x5, 40(x6)	$\text{Memoria}[x6 + 40] = x5$	Spostamento di una parola da registro a memoria.
Trasferimento dati	lh x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di una mezza parola da memoria a registro.
Trasferimento dati	lhu x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di una mezza parola senza segno da memoria a registro.
Trasferimento dati	sh x5, 40(x6)	$\text{Memoria}[x6 + 40] = x5$	Spostamento di una mezza parola da registro a memoria.
Trasferimento dati	lb x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di un byte da memoria a registro.
Trasferimento dati	lbu x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di un byte senza segno da memoria a registro.
Trasferimento dati	sb x5, 40(x6)	$\text{Memoria}[x6 + 40] = x5$	Spostamento di un byte da registro a memoria.
Trasferimento dati	lr.d x5, (x6)	$x5 = \text{Memoria}[x6]$	Caricamento di una parola come prima fase di un'operazione atomica sulla memoria.
Trasferimento dati	sc.d x7, x5, (x6)	$\text{Memoria}[x6] = x5; x7 = 0/1$	Memorizzazione di una parola come seconda fase di un'operazione atomica sulla memoria.
Trasferimento dati	lui x5, 0x12345	$x5 = 0x12345000$	Caricamento di una costante su 20 bit nei 12 bit più significativi di una parola.

6.1 Operandi RISC-V

6.2 Istruzioni Aritmetiche e di Trasferimento Dati

6.3 Linguaggio Assembler: Istruzioni Logiche, Scorrimento e Salti

Tipo di istruzioni	Istruzioni (Esempio)	Significato	Commenti
Logiche	and x5, x6, x7	$x5 = x6 \& x7$	Operandi in tre registri; AND bit a bit.
Logiche	or x5, x6, x8	$x5 = x6 x8$	Operandi in tre registri; OR bit a bit.
Logiche	xor x5, x6, x9	$x5 = x6 \wedge x9$	Operandi in tre registri; XOR bit a bit.
Logiche	andi x5, x6, 20	$x5 = x6 \& 20$	AND bit a bit tra operando in registro e costante.
Logiche	ori x5, x6, 20	$x5 = x6 20$	OR bit a bit tra operando in registro e costante.
Logiche	xori x5, x6, 20	$x5 = x6 \wedge 20$	XOR bit a bit tra operando in registro e costante.
Scorrimento	sll x5, x6, x7	$x5 = x6 \ll x7$	Scorrimento a sinistra mediante registro.
Scorrimento	srl x5, x6, x7	$x5 = x6 \gg x7$	Scorrimento a destra mediante registro.
Scorrimento	sra x5, x6, x7	$x5 = x6 \ggg x7$	Scorrimento a destra aritmetico mediante registro.
Scorrimento	slli x5, x6, 3	$x5 = x6 \ll 3$	Scorrimento a sinistra mediante costante.
Scorrimento	srli x5, x6, 3	$x5 = x6 \gg 3$	Scorrimento a destra mediante costante.
Scorrimento	srai x5, x6, 3	$x5 = x6 \ggg 3$	Scorrimento a destra aritmetico mediante costante.
Salti condiz.	beq x5, x6, 100	Se $(x5 == x6)$ vai a $PC + 100$	Test di uguaglianza; salto relativo al PC.
Salti condiz.	bne x5, x6, 100	Se $(x5 \neq x6)$ vai a $PC + 100$	Test di disuguaglianza; salto relativo al PC.
Salti condiz.	blt x5, x6, 100	Se $(x5 < x6)$ vai a $PC + 100$	Comparazione di minoranza; salto relativo al PC.
Salti condiz.	bge x5, x6, 100	Se $(x5 \geq x6)$ vai a $PC + 100$	Comparazione di maggioranza o uguaglianza.
Salti condiz.	bltu x5, x6, 100	Se $(x5 < x6)$ vai a $PC + 100$	Comparazione di minoranza senza segno.
Salti condiz.	bgeu x5, x6, 100	Se $(x5 \geq x6)$ vai a $PC + 100$	Comparazione di maggioranza o ug. senza segno.
Salti incondiz.	jal x1, 100	$x1 = PC + 4$; vai a $PC + 100$	Chiamata a procedura con indirizzo rel. al PC.
Salti incondiz.	jalr x1, 100(x5)	$x1 = PC + 4$; vai a $x5 + 100$	Ritorno da procedura; chiamata indiretta.

6.4 Dettaglio sulle Operazioni Logiche e di Scorrimento

I calcolatori moderni operano anche a livello di bit per ottimizzare i calcoli e gestire maschere di dati.

Shift sinistro (slli): Sposta i bit a sinistra inserendo degli 0 nelle posizioni libere. Equivale alla moltiplicazione:

$$x_{11} = x_{19} \cdot 2^k$$

Esempio: $9 \cdot 2^4 = 144$. Attenzione alla gestione dell'overflow.

Shift destro logico (srli): Sposta i bit a destra inserendo 0. Equivale alla divisione intera per 2^k .

Shift destro aritmetico (srai): Sposta i bit a destra mantenendo il bit di segno (estensione del segno).

Operatori Bit-a-bit:

- **AND:** Utilizzato per azzerare bit specifici tramite una maschera.
- **OR:** Utilizzato per impostare bit specifici a 1.
- **XOR:** Risulta 1 se i bit sono diversi.

Trucchi comuni:

- Azzerare un registro: `xor x9, x9, x9` $\Rightarrow x9 = 0$.
- Operazione NOT: $NOT(A) = A \oplus 1$ (eseguito bit a bit).

6.5 Istruzioni per il Controllo del Flusso (Decisioni)

Il RISC-V utilizza salti condizionati e incondizionati per implementare strutture di controllo come `if`, `loops` e `switch`.

Traduzione di una clausola If-Else: Codice C:

```
if (i == j)
    f = g + h;
else
    f = g - h;
```

Codice RISC-V corrispondente (assumendo `i` in `x22`, `j` in `x23`, `f` in `x19`, `g` in `x20`, `h` in `x21`):

```
bne x22, x23, ELSE    # Se i != j salta a ELSE
add x19, x20, x21      # f = g + h
beq x0, x0, END        # Salto incondizionato a END
ELSE: sub x19, x20, x21 # f = g - h
END:
```

Cicli e Blocchi di Base: I cicli vengono implementati tornando indietro a un'etichetta tramite salti. Un **blocco di base** è una sequenza di istruzioni senza salti (tranne alla fine) e senza etichette di destinazione (tranne all'inizio).

Switch/Case: Queste strutture vengono spesso implementate tramite **tabelle di indirizzi** (jump tables) e l'istruzione di salto indiretto `jalr`, che permette di saltare a un indirizzo calcolato dinamicamente.

7 Gestione delle Procedure e della Memoria in RISC-V

7.1 Procedure e Protocollo

- L'utilità di un computer sarebbe molto limitata se non si potessero chiamare procedure (o funzioni).
- Possiamo pensare a una procedura come a una "scatola nera" che esegue un certo task, senza che ne conosciamo necessariamente i dettagli.
- La cosa importante è definire chiaramente un protocollo (o convenzione) di chiamata alle procedure.
- Le fasi del protocollo prevedono di:
 1. Caricare i parametri di input della procedura in posti noti a priori.
 2. Trasferire il controllo alla procedura, il che include acquisire le risorse necessarie, eseguire il compito, caricare i valori di ritorno in posti noti e restituire il controllo al chiamante.
 3. Prendere il valore di ritorno della procedura e "cancellare" le tracce (pulire registri, ecc.).
- Il modo in cui questo protocollo viene messo in pratica dipende dall'architettura e dalle convenzioni di chiamata del compilatore. Nel nostro caso, ci limitiamo a cenni sull'architettura RISC-V.

7.2 Protocollo di chiamata RISC-V

- L'idea di base è di cercare di usare laddove possibile i registri, perché sono il meccanismo più veloce per effettuare il passaggio dei parametri.
- Convenzioni del RISC-V:
 - I registri `x10-x17` sono usati per i parametri in ingresso e per i valori di ritorno.
 - Il registro `x1` viene utilizzato come indirizzo di ritorno per tornare al punto di partenza (chiamato anche `ra`, ovvero "return address").

7.2.1 Le Istruzioni di Salto

- L'indirizzo `x1` viene usato in particolare nelle istruzioni di "jump and link" (`jal`) e "jump and link register" (`jalr`). Queste istruzioni effettuano il salto e contemporaneamente memorizzano in `x1` l'indirizzo di ritorno.
- L'indirizzo salvato in `x1` è pari a `PC (program counter) + 4`, che corrisponde all'istruzione successiva alla `jal/jalr`.
- Esempi di istruzioni:
 - `jal x1, 100` (Salta e collega): memorizza `x1 = PC+4` e va a `PC+100`.
 - `jalr x1, 100(x5)` (Salta e collega mediante registro): memorizza `x1 = PC+4` e va a `x5+100`.
 - Alla fine della procedura sarà sufficiente fare un salto del tipo: `jalr x0, 0(x1)`.

7.3 Uso dello Stack e Chiamate a Procedura in C

- In certi casi i registri non bastano perché la procedura ha più parametri di quanti sono i registri a disposizione.
- In tal caso si usa uno stack (pila). Questa è una struttura dati gestita in modalità LIFO in cui è possibile caricare i parametri in più a partire da una posizione nota alla procedura (puntata dal registro `x2`, chiamato anche `sp`, o stack pointer).
- Lo stack si può usare anche per caricare variabili locali tramite un'operazione di push e per salvare dei valori di registri che dovranno essere ripristinati in seguito.
- Alla fine della procedura si può ripulire lo stack (operazione pop) riportandolo alla situazione precedente.

7.3.1 Esempio e Fasi di Compilazione

Consideriamo il seguente codice in C:

```
long long int esempio_foglia(long long int g, long long int h,  
                             long long int i, long long int j) {  
    long long int f;  
    f = (g+h) - (i+j);  
    return f;  
}
```

Come viene tradotta questa procedura?

- **Parametri di input e variabili:** $g = x10$; $h = x11$; $i = x12$; $j = x13$. La variabile locale $f = x20$. Usiamo anche i registri temporanei `x5` e `x6`, assumendo di dover salvare `x5`, `x6` e `x20` nello stack.

Prologo:

- Per prima cosa il compilatore sceglie un'etichetta associata all'indirizzo di entrata della procedura. In fase di linking l'etichetta sarà collegata a un indirizzo reale.
- Si salvano in memoria tutti i registri usati in modo da poterli ripristinare. Questa fase è chiamata prologo e potrebbe richiedere di allocare nello stack anche spazio per le variabili locali.
- *Esempio di codice Assembly (Prologo):*
 - `esempio_foglia: addi sp, sp, -24` (decrementa `sp` di 24 per far posto a 3 double word).
 - `sd x5, 16(sp)` (salvataggio di `x5` in `sp+16`).
 - `sd x6, 8(sp)` (salvataggio di `x6` in `sp+8`).
 - `sd x20, 0(sp)` (salvataggio di `x20` in `sp+0`).

Esecuzione ed Epilogo:

- Dopo il prologo, avviene l'esecuzione della procedura vera e propria:

- `add x5, x10, x11` (in `x5` salviamo `g+h`).
- `add x6, x12, x13` (in `x6` salviamo `i+j`).
- `sub x20, x5, x6` (in `x20`: $(g+h)-(i+j)$).
- Segue l'epilogo per ripristinare i registri usati e settare il valore di ritorno:
 - `addi x10, x20, 0` (trasferisce `x20` in `x10` come valore di ritorno).
 - `ld x20, 0(sp)` / `ld x6, 8(sp)` / `ld x5, 16(sp)` (ripristino dei registri prelevando i valori esattamente da dove li avevamo messi).
 - `addi sp, sp, 24` (ripristino dello stack; tutto ciò che è sotto `sp` non è significativo).
 - `jalr x0, 0(x1)` (salto al punto da dove siamo arrivati).

7.3.2 Un bagno di realtà

- Nessun compilatore farebbe mai questo...
- Infatti, i registri temporanei `x5` e `x6` in realtà non devono essere salvati durante la chiamata (avremmo potuto evitare due `ld` e due `sd`).
- RISC-V adotta la seguente convenzione:
 - `x5-x7` e `x28-x31`: registri temporanei, che non sono salvati in caso di chiamata a procedura.
 - `x8-x9` e `x18-x27`: registri da salvare il cui contenuto deve essere preservato in caso di chiamata a procedura.

7.4 Procedure Ricorsive e Ottimizzazioni

- In realtà le cose possono complicarsi per via di variabili locali e procedure annidate. Questo si risolve usando sempre lo stack e allocando al suo interno variabili locali e valori del registro di ritorno `x1`.

7.4.1 Esempio Pratico: La funzione Fattoriale

```
long long int fact(long long int n) {
    if (n<1) return 1;
    else return n*fact(n-1);
}
```

- In chiamate annidate (come la procedura ricorsiva appena vista), abbiamo il problema della sovrascrittura di `x1` (che rende impossibile il ritorno) e `x10` (usato per il parametro `n`).
- Soluzione: salvaguardare i registri `x1`, `x10` creando spazio nello stack (es. `addi sp, sp, -16`).

7.4.2 Tail Recursion (Ricorsione di coda)

- Per quanto le ricorsioni siano eleganti, spesso sono causa di inefficienze (“Never hire a programmer who solves the factorial with a recursion!”).
- Consideriamo una funzione `sum(n, acc)` modificata affinché la chiamata ricorsiva sia l’ultima cosa che la procedura fa. Questa ricorsione viene detta “di coda”.
- In una ricorsione classica, ogni frame di attivazione deve essere preservato per produrre il risultato corretto tornando su per la catena di chiamate.
- Nella ricorsione di coda, conservare i dati nel frame di attivazione è inutile: si può usare un unico frame di attivazione e svolgere la ricorsione in iterazione.
- Alcuni compilatori sanno riconoscere la ricorsione di coda: evitano che il chiamante salvi `ra` per nulla sostituendo la `jal` con un salto incondizionato (es. `beq x0, x0, sum`), come fa il gcc con `-O2`.

7.5 Memoria e Variabili

7.5.1 Classi di Memoria e Record di Attivazione

- Le variabili in C sono associate a locazioni di memoria caratterizzate per tipo e *storage class*:
 - **Automatic**: variabili locali con ciclo di vita collegato alla funzione.
 - **Static**: sopravvivono alle chiamate di procedura. Nel RISC-V, questa zona è accessibile attraverso il registro `x3`, chiamato anche `gp` (“global pointer”).
- Le variabili locali, quando sono tante, si allocano nello stack.
- Il segmento di stack che contiene registri salvati e variabili locali relative a una procedura viene chiamato **record di attivazione** (o *stack frame*).
- Poiché il registro `sp` cambia (cresce verso il basso) durante l’esecuzione, alcuni programmi utilizzano il registro `x8`, chiamato `fp` (“frame pointer”), mantenuto costante durante la funzione. L’organizzazione standard tra `FP` (indirizzi alti) e `SP` (indirizzi bassi) prevede registri a scorrimento, indirizzo di ritorno, registri salvati, e vettori/strutture locali in fondo.

7.5.2 Schema Generale di Memoria RISC-V

In aggiunta a variabili globali e locali, abbiamo le variabili dinamiche (allocate con `malloc/free`, `new/delete`), salvate nell’**heap**.

- Lo **Stack** procede per indirizzi decrescenti.
- L’**Heap** procede per indirizzi crescenti.
- Sotto l’heap vi sono i **Dati statici**, il segmento di **Testo** (le istruzioni, PC) e una zona riservata.

Nome	N. Registro	Utilizzo	Da conservare?
x0	0	Costante 0	n.a.
x1 (ra)	1	Registro di ritorno (collegamento)	sì
x2 (sp)	2	Stack pointer	sì
x3 (gp)	3	Global pointer	sì
x4 (tp)	4	Thread pointer	sì
x5-x7	5-7	Variabili temporanee	no
x8-x9	8-9	Variabili da preservare	sì
x10-x17	10-17	Argomenti/risultati	no
x18-x27	18-27	Variabili da preservare	sì
x28-x31	28-31	Variabili temporanee	no

7.5.3 Riassunto Convenzioni Registri RISC-V

8 Esempi di Programmi Assembly RISC-V

8.1 Informazioni Generali e Scopo della Lezione

Il titolo della lezione è *Esempi di programmi assembly RISC-V* ed è tenuta da Giovanni Iacca, basandosi su materiale dei Prof. Luigi Palopoli, Marco Roveri e Luca Abe-ni. L'obiettivo è illustrare alcuni esempi e frammenti di programmi scritti in assembly RISC-V. Nel prosieguo del corso verranno trattati anche gli assembly INTEL e ARM per comprenderne le differenze.

8.2 Semplici Istruzioni Aritmetiche e Logiche

Il codice di partenza è $f = (g + h) - (i + j)$;

8.2.1 Traduzione Base

Si suppone che le variabili g , h , i , j si trovino rispettivamente nei registri x19, x20, x21, x22, e che il risultato debba essere salvato in x23. Il codice assembly corrispondente è:

```
add x5, x19, x20
add x6, x21, x22
sub x23, x5, x6
```

8.2.2 Traduzione Ottimizzata (v2)

È possibile utilizzare un registro in meno, salvando il risultato intermedio direttamente in x23. Il codice diventa:

```
add x23, x19, x20
add x6, x21, x22
sub x23, x23, x6
```

8.3 Accesso alla Memoria

Il codice di partenza è $a[12] = h + a[8]$; Si ipotizza che h si trovi in $x21$ e che l'indirizzo base del vettore a sia nel registro $x22$. La traduzione RISC-V prevede tre passi:

- Lettura dalla memoria: `ld x9, 64(x22)` (carica $a[8]$ in $x9$).
- Addizione: `add x9, x21, x9` (calcola $h + a[8]$).
- Scrittura in memoria: `sd x9, 96(x22)` (salva il risultato in $a[12]$).

8.4 Blocchi Condizionali e Cicli

8.4.1 Condizione di Uguaglianza (`if (i == j)`)

Se vero: $f = g + h$; Se falso (else): $f = g - h$; Viene utilizzato il salto condizionato `bne x22, x23, L2` per saltare all'etichetta L2 se i registri sono diversi. In caso contrario, si esegue l'addizione e si salta la parte "else" con `beq x0, x0, L3`.

8.4.2 Condizione di Disuguaglianza (`if (i < j)`)

Se vero: $f = g + h$; Altrimenti: $f = g - h$; Può essere tradotto usando `slt` e `beq`, oppure in modo più efficiente con l'istruzione `blt x22, x23, L2` per saltare direttamente se $x22 < x23$.

8.4.3 Ciclo While (`while (a[i] == k)`)

Si calcola l'indirizzo dell'elemento corrente moltiplicando l'indice per 8 tramite `slli x10, x22, 3`. Si esce dal ciclo se il valore non è uguale a k usando `bne x9, x24, L2`. Si riavvia il ciclo incondizionatamente con `beq x0, x0, L1`.

8.5 Funzioni Foglia e Non Foglia

8.5.1 Funzioni Foglia

Sono funzioni che non ne richiamano altre al loro interno. Senza ottimizzazioni, è necessario salvare e ripristinare i registri nello stack (prologo ed epilogo), allocando spazio con `addi sp, sp, -24` e deallocandolo con `addi sp, sp, 24`. Con le ottimizzazioni (e se non è necessario preservare registri temporanei), le operazioni sullo stack possono essere omesse, ritornando direttamente al chiamante con `jalr x0, 0(x1)`.

8.5.2 Tabella dei Registri RISC-V

I registri hanno nomi specifici e convenzioni d'uso:

- $x0$ (zero): Costante 0.
- $x1$ (ra): Indirizzo di ritorno, salvato dal chiamante.
- $x2$ (sp): Stack pointer, salvato dal chiamato.
- $x10$ - $x11$ (a0-a1): Argomenti di funzione o valori restituiti.

8.5.3 Funzioni Non Foglia

Una funzione non foglia, come una che richiama `inc(x)`, richiede il salvataggio del registro `ra` (indirizzo di ritorno) sullo stack. Esempio GCC con ottimizzazione (O1): estende lo stack (`addi sp, sp, -16`), salva `ra` (`sd ra, 8(sp)`), richiama la sottofunzione (`call inc`), e poi ripristina `ra` e lo stack prima di uscire con `ret`.

8.6 Algoritmi Complessi in Assembly

8.6.1 Ordinamento di Array (Insertion Sort)

Viene mostrato un esempio con due funzioni: `ordina` e `sposta`. La funzione `sposta` prevede calcoli complessi sugli indirizzi di memoria, decrementi d'indice (`addiw a1, a1, -1`) e verifiche condizionali multiple all'interno di un ciclo `while` (`bge, bltz`) per scambiare gli elementi dell'array. La funzione `ordina` gestisce un loop esterno, richiamando `sposta` ad ogni iterazione, comportandosi quindi come funzione non foglia e gestendo correttamente lo stack (`addi sp, sp, -32, sd ra, 24(sp)`).

8.6.2 Copia di Stringhe

Basato sul loop C: `while ((d[i]=s[i])!='\0') i+=1;`. Tradotto in assembly RISC-V utilizzando istruzioni di accesso al singolo byte della memoria: `lbu x6, 0(x5)` per leggere un carattere e `sb x6, 0(x7)` per scriverlo nella nuova posizione. La condizione di uscita verifica se il byte appena copiato equivale a zero (`beq x6, x0, LoopCopiaStringaEnd`).

9 Tutorato Assembly RISC-V

9.1 Concetti Teorici Fondamentali

- **Registri RISC-V:** L'architettura base a 64 bit dispone di **32 registri** general-purpose da 64 bit (ovvero *double word*).
- **Registro x0:** È un registro speciale cablato al valore costante **0**. Ogni tentativo di modifica viene ignorato; si rivela estremamente utile per semplificare diverse operazioni (es. copia di registri o salti incondizionati).
- **Formati delle Istruzioni:** Il **Formato I** (Immediato) è impiegato per operazioni che richiedono un operando costante (come l'istruzione `addi`) e per gli accessi in memoria (`load`), permettendo di codificare la costante direttamente nei 32 bit dell'istruzione.
- **Endianness:** Il processore RISC-V utilizza la convenzione **Little Endian**, memorizzando il byte meno significativo all'indirizzo di memoria più basso.

9.2 Traduzione di Costrutti: dal C all'Assembly

Operazioni Aritmetiche, Logiche e Shift

- *Espressioni composte* (es. `a = (b + c) - d`): Si utilizzano registri temporanei per memorizzare i calcoli intermedi.

```
add x5, x11, x12 % x5 = b + c (risultato intermedio)
sub x10, x5, x13 % a = x5 - d
```

- *Moltiplicazioni per potenze di 2* (es. $a = b * 8$): Si ottimizza il calcolo utilizzando l'istruzione di **shift logico a sinistra** (`slli`). Moltiplicare per 8 (2^3) equivale a traslare i bit di 3 posizioni.

```
slli x10, x11, 3 % a = b << 3 (equivale ad a = b * 8)
```

Accesso alla Memoria e Array

- *Calcolo dell'offset*: L'indirizzamento in RISC-V è al byte. Lavorando con array di *double word* (8 byte), l'indice deve essere sempre moltiplicato per 8 per ricavare lo spiazzamento corretto.
- *Esempio di traduzione* ($A[4] = h + A[2]$):

```
ld x9, 16(x22) % Carica A[2] (indice 2 * 8 = offset 16)
add x9, x21, x9 % Somma h con A[2]
sd x9, 32(x22) % Salva il risultato in A[4] (indice 4 * 8 = offset 32)
```

Costrutti Condizionali (if-else)

- *La regola della condizione opposta*: Per tradurre i costrutti di selezione, la prassi comune è valutare la **condizione opposta** rispetto a quella del codice ad alto livello, in modo da poter saltare l'esecuzione del blocco `if` qualora la condizione sia falsa.
- *Esempio if-else* (`if (i < j) f = g + h; else f = g - h;`):

```
bge x22, x23, ELSE % Condizione opposta: se i >= j salta a ELSE
add x19, x20, x21 % Ramo IF: f = g + h
beq x0, x0, ESCI % Salto incondizionato alla fine
ELSE:
sub x19, x20, x21 % Ramo ELSE: f = g - h
ESCI: % Fine blocco
```

Costrutti Iterativi (while)

- *Gestione dei cicli su Array* (`while (salva[i] == k) i += 1;`): I cicli richiedono il calcolo dinamico dell'indirizzo base ad ogni singola iterazione, seguito da un controllo sulla condizione di uscita.

```
CICLO:
slli x10, x22, 3 % Moltiplica indice i per 8 per l'offset in byte
add x10, x10, x25 % Aggiunge l'offset all'indirizzo base dell'array
ld x9, 0(x10) % Carica fisicamente salva[i] dalla memoria
bne x9, x24, ESCI % Condizione d'uscita: se salva[i] != k, interrompi
addi x22, x22, 1 % Incrementa l'indice i di 1
beq x0, x0, CICLO % Salto incondizionato per ripetere
ESCI:
```

10 Soluzioni Mockup Complete

1. **Domanda:** Il record di attivazione in RISC-V contiene diverse informazioni quando viene chiamata una funzione, questo include:

- (A) Registri a scorrimento salvati, Registri del tipo “callee-saved” salvati, Indirizzo di ritorno, Registri non a scorrimento salvati, Registri del tipo “caller-saved” salvati, ed altre informazioni.
- (B) Registri a scorrimento salvati, Registri del tipo “caller-saved” salvati, Indirizzo di ritorno, ed altre informazioni.
- (C) Registri non a scorrimento salvati, Registri del tipo “callee-saved” salvati, ed altre informazioni.
- (D) Nessuna delle risposte precedenti è corretta.

Risposta corretta: (A)

Perché è corretta: Nelle convenzioni di chiamata del RISC-V, il record di attivazione (Stack Frame) è la porzione di stack dedicata a una procedura per memorizzare tutte le informazioni necessarie al suo funzionamento. L’opzione A è l’unica che fornisce una lista completa, includendo sia i registri che devono essere preservati dal chiamato (*callee-saved*), sia quelli preservati dal chiamante (*caller-saved*), sia il fondamentale indirizzo di ritorno (*ra*).

Perché scartare le altre: L’opzione (B) omette i registri *callee-saved*, che sono cruciali. L’opzione (C) omette l’indirizzo di ritorno e i *caller-saved*. L’opzione (D) è errata poiché la (A) è esaustiva.

2. **Domanda:** Cosa contiene il registro *x5* dopo le seguenti istruzioni?

```
addi x5, x0, 10
addi x6, x0, 21
sll x5, x5, x6
```

- (A) 0000 0000 0001 0100 0000 0000 0000 0000 0000
- (B) 0000 0000 0000 0000 0000 0000 0000 0000 0000
- (C) 0000 0000 0000 0000 0000 0000 0000 0000 1010
- (D) 0000 0000 0000 0001 0100 0000 0000 0000 0000
- (E) Nessuna delle risposte precedenti è corretta.

Risposta corretta: (A)

Perché è corretta: Il registro *x5* viene inizializzato a 10 (in binario 1010). L’istruzione *sll* (Shift Left Logical) scorre i bit di *x5* verso sinistra del valore contenuto in *x6* (21 posizioni), riempiendo i bit meno significativi con zeri. Questo equivale a calcolare 10×2^{21} . Spostando la sequenza 1010 di 21 posizioni, otteniamo 0001 0100 seguito da esattamente 20 zeri (cinque blocchi da 4 zeri).

Perché scartare le altre: La (B) presuppone erroneamente che il risultato sia zero. La (C) mostra il valore iniziale di *x5* (10) senza alcuno shift. La (D) ha un numero errato di zeri finali (ne ha solo 16, corrispondenti a uno shift di 17 posizioni anziché 21).

3. **Domanda:** Secondo le convenzioni di chiamata del RISC-V illustrate a lezione, quali registri vengono comunemente utilizzati per il passaggio dei parametri in ingresso alle procedure e per i valori di ritorno?

- (A) Da `x1` a `x8`
- (B) Da `x10` a `x17`
- (C) Da `x18` a `x27`
- (D) Da `x28` a `x31`
- (E) Nessuna delle risposte precedenti è corretta.

Risposta corretta: (B)

Perché è corretta: L'interfaccia binaria (*ABI*) del RISC-V dedica in modo standard i registri da `x10` a `x17` (noti come `a0-a7`) al passaggio dei parametri in ingresso. I primi due (`x10` e `x11` / `a0` e `a1`) sono utilizzati anche per restituire i valori di ritorno.

Perché scartare le altre: L'intervallo (A) include registri con scopi diversi (es. `x1` è l'indirizzo di ritorno `ra`, `x2` è lo stack pointer `sp`). L'intervallo (C) racchiude i registri salvati (`s2-s11`). L'intervallo (D) racchiude i registri temporanei (`t3-t6`).

4. **Domanda:** In merito all'ottimizzazione del codice RISC-V, in cosa consiste l'ottimizzazione per il compilatore nota come "ricorsione di coda" (tail recursion)?

- (A) Nel salvataggio sistematico di tutti i registri nello stack prima di effettuare qualsiasi chiamata ricorsiva.
- (B) Nell'uso esclusivo dei registri callee-saved per evitare conflitti con la funzione chiamante.
- (C) Nella sostituzione di un'istruzione `jal` seguita da un ritorno, con un salto incondizionato alla procedura, semplificando la gestione dello stack.
- (D) Nell'esecuzione parallela di più rami della ricorsione utilizzando le pipeline della CPU.
- (E) Nessuna delle risposte precedenti è corretta.

Risposta corretta: (C)

Perché è corretta: La *tail recursion* si verifica quando la chiamata ricorsiva è l'ultima istruzione eseguita da una funzione. Il compilatore la ottimizza evitando di creare un nuovo record di attivazione: sostituisce la chiamata a funzione (`jal`) con un semplice salto al prologo (`j`), riutilizzando lo stack frame attuale e prevenendo lo *stack overflow*.

Perché scartare le altre: La (A) descrive l'esatto opposto (spreco di stack). La (B) riguarda una strategia di allocazione dei registri, non l'ottimizzazione ricorsiva. La (D) descrive parallelismo a livello hardware (ILP), slegato dal concetto logico di tail recursion.

5. **Domanda:** Riporta quale di queste istruzioni è corretta per completare la traduzione del frammento (sostituendo `X1`): `while (z < g) { f = f + j; z = z + 1; }`

- (A) `add x11, x11, x12`

- (B) `add x12, x12, x11`
- (C) `addi x12, x12, 1`
- (D) `add x13, x12, x11`
- (E) Nessuna delle risposte precedenti è corretta.

Risposta corretta: (B)

Perché è corretta: Nel C, l'operazione principale del ciclo è $f = f + j$. Secondo l'ABI passata come argomento, f (terzo parametro) si trova nel registro `x12` (**a2**) e j (secondo parametro) in `x11` (**a1**). L'istruzione per sommarli salvando il risultato in f è `add x12, x12, x11`.

Perché scartare le altre: La (A) salva il risultato in j anziché in f . La (C) aggiunge 1 a f , ignorando la variabile j . La (D) sovrascrive g (`x13`) distruggendo la condizione del ciclo.

6. **Domanda:** Quale delle seguenti istruzioni deve sostituire `X1` affinché il calcolo dell'indirizzo di memoria dell'array sia corretto? (`long long int v[]`)

- (A) `slli x5, x11, 2`
- (B) `add x5, x11, x0`
- (C) `slli x5, x10, 3`
- (D) `slli x5, x11, 3`

Risposta corretta: (D)

Perché è corretta: Il vettore è di tipo `long long int`, i cui elementi occupano 8 byte ciascuno. Per trovare l'indirizzo dell'elemento i -esimo (in `x11`), l'indice deve essere moltiplicato per 8 (il *peso* in byte). Lo shift logico a sinistra di 3 posizioni (`slli ... 3`) equivale a moltiplicare l'indice per $2^3 = 8$.

Perché scartare le altre: La (A) moltiplica per 4, il che sarebbe corretto solo per un normale `int` a 32 bit. La (B) non moltiplica l'indice, assumendo erroneamente un array di `char` (1 byte). La (C) moltiplica l'indirizzo base (`x10`) invece dell'indice, distruggendo del tutto il puntatore alla memoria.

7. **Domanda:** Qual è lo scopo principale dell'istruzione `addi sp, sp, 32` trovata in un epilogo?

- (A) Allocare 32 byte di memoria nello stack per salvare i registri locali in vista di un'ulteriore chiamata.
- (B) Deallocare lo spazio precedentemente riservato nello stack (Record di Attivazione) prima di ritornare al chiamante.
- (C) Caricare i valori di ritorno nel registro `sp`.
- (D) Passare 32 byte di parametri dal callee al caller.

Risposta corretta: (B)

Perché è corretta: In RISC-V lo stack cresce verso indirizzi di memoria *decrescenti*. Quando una funzione inizia (prologo), sottrae spazio da `sp` per fare posto. Al termine (epilogo), deve ripristinare il puntatore originale sommando esattamente lo stesso valore (+32), "deallocando" logicamente lo spazio per le future chiamate.

Perché scartare le altre: La (A) confonde l'epilogo con il prologo (l'allocazione

si fa con un valore negativo). La (C) è errata perché i valori di ritorno vanno in `a0/a1`, non nello Stack Pointer. La (D) è priva di senso a livello architetturale.

8. **Domanda:** A quale istruzione base corrisponde la pseudo-istruzione `ret`?

- (A) `jal x0, ra`
- (B) `jalr x0, 0(x1)`
- (C) `jr x1`
- (D) `beq x0, x0, x1`

Risposta corretta: (B)

Perché è corretta: La pseudo-istruzione `ret` serve a saltare di ritorno all'indirizzo memorizzato dal chiamante. Il registro standard che contiene questo indirizzo è `x1` (`ra`). A livello hardware, l'istruzione `jalr x0, 0(x1)` salta incondizionatamente all'indirizzo in `x1` (con offset 0) e salva l'istruzione successiva in `x0` (scartandola, poiché `x0` è cablato a 0).

Perché scartare le altre: La (A) usa `jal`, che si basa su un offset relativo al Program Counter e non può saltare a un indirizzo contenuto in un registro. La (C) è a sua volta un'altra pseudo-istruzione (alias di `jalr`). La (D) è un'istruzione di *branch* (salto condizionato), inadatta a salti assoluti basati su registri.

9. **Domanda:** Lavorando con un'architettura RISC-V a 64 bit (RV64), qual è la differenza fondamentale tra `lw` e `lwu`?

- (A) `lw` carica 64 bit dalla memoria, mentre `lwu` ne carica solo 32.
- (B) Entrambe caricano 32 bit dalla memoria, ma `lw` estende il segno sui restanti 32 bit del registro a 64 bit, mentre `lwu` riempie i restanti 32 bit con zeri.
- (C) `lw` carica un dato dalla memoria e lo allinea a sinistra nel registro, `lwu` lo allinea a destra.
- (D) Non esiste alcuna differenza a livello hardware, l'assemblatore sceglie quale usare in base al tipo di dato.

Risposta corretta: (B)

Perché è corretta: Una "word" in RISC-V è di 32 bit. Poiché i registri di RV64 sono a 64 bit, quando si caricano 32 bit bisogna decidere cosa fare della metà superiore del registro. `lw` preserva il valore matematico dei numeri negativi copiando il bit di segno in tutte le posizioni superiori (*sign extension*). `lwu` (Unsigned) tratta il dato come positivo, riempiendo la parte alta semplicemente con zeri (*zero extension*).

Perché scartare le altre: La (A) è errata perché il caricamento a 64 bit si fa con `ld` (Load Doubleword). La (C) è falsa (tutti i valori sono allineati al Least Significant Bit a destra). La (D) è falsa perché `lw` e `lwu` corrispondono a *opcode* binari hardware fisicamente distinti.

10. **Domanda:** Nelle istruzioni di salto condizionato (come `beq`), come viene calcolato a livello hardware l'indirizzo di memoria dell'istruzione a cui saltare?

- (A) Tramite un indirizzamento assoluto: l'indirizzo a 32/64 bit viene caricato direttamente da un registro base.

- (B) L'indirizzo target viene letto dallo Stack Pointer (**sp**).
- (C) Tramite indirizzamento PC-relative: l'indirizzo target si ottiene sommando il valore attuale del Program Counter (PC) con un offset codificato direttamente all'interno dell'istruzione di salto.
- (D) Il target viene calcolato moltiplicando per 4 il valore contenuto nel registro **ra**.

Risposta corretta: (C)

Perché è corretta: I salti condizionati vengono generalmente utilizzati per saltare a blocchi di codice vicini (**if**, **loop**). RISC-V ottimizza questa operazione utilizzando l'indirizzamento *PC-relative*: calcola la distanza (offset) tra l'istruzione corrente e la destinazione e la somma al Program Counter (PC). Questo permette di risparmiare spazio, poiché un piccolo offset entra comodamente nei 32 bit dell'istruzione.

Perché scartare le altre: La (A) descrive il meccanismo di **jalr**, usato tipicamente per i ritorni a funzione o chiamate tramite puntatore, non per i branch. La (B) coinvolge lo **stack**, che contiene i dati locali e non le istruzioni da eseguire. La (D) non ha senso architetturale.

11. **Domanda:** Quale istruzione deve sostituire **X1** per permettere l'avanzamento logico corretto del ciclo in **sumV**? (***v = *u + 1;**)

- (A) **lw a5, 0(a0)**
- (B) **lw a1, 0(a6)**
- (C) **lw a5, 4(a0)**
- (D) **lw a5, -4(a0)**

Risposta corretta: (A)

Perché è corretta: L'istruzione in C ***v = *u + 1** impone prima di tutto di leggere in memoria il valore puntato da **u** per portarlo nel processore. Sapendo che il puntatore **u** è mappato sul registro **a0**, l'istruzione corretta per dereferenziarlo (senza alcuno spiazamento, dato che il puntatore indica esattamente l'elemento attuale) è **lw a5, 0(a0)**, che carica il valore nel registro di appoggio **a5**.

Perché scartare le altre: La (B) usa i registri completamente sbagliati. La (C) usa un offset positivo di 4, saltando inavvertitamente il primo elemento dell'array prima ancora di leggerlo. La (D) usa un offset negativo, leggendo fuori dai limiti previsti dell'array.

12. **Domanda:** Quale combinazione deve sostituire **X1**, **X2**, **X3** per tradurre la copia di **char** e il **break** su **'\0'**?

- (A) **X1: lw x7, 0(x6) | X2: sw x7, 0(x28) | X3: beq x7, x0, end**
- (B) **X1: lbu x7, 0(x6) | X2: sb x7, 0(x28) | X3: bne x7, x0, end**
- (C) **X1: lbu x7, 0(x6) | X2: sb x7, 0(x28) | X3: beq x7, x0, end**
- (D) **X1: lbu x6, 0(x7) | X2: sb x28, 0(x7) | X3: beq x7, x0, end**

Risposta corretta: (C)

Perché è corretta: Trattandosi di stringhe, i dati sono di tipo **char** (1 byte). Occorre caricare un singolo byte senza segno (**lbu**), salvarlo in destinazione (**sb**) e infine verificare la condizione di uscita. Il C dice **if (c == '\0') break;**, quindi

dobbiamo uscire dal ciclo (saltare a **end**) **se** il carattere in **x7** è **uguale** a zero. L'istruzione per farlo è **beq x7, x0, end**.

Perché scartare le altre: La (A) usa **lw** e **sw**, che spostano blocchi di 4 byte (parole intere) distruggendo la memoria adiacente alla stringa. La (B) esce dal ciclo se il carattere **non** è zero (**bne**), interrompendo la copia al primo carattere valido. La (D) inverte i registri per i puntatori, sovrascrivendo gli indirizzi di memoria.

13. **Domanda:** Calcola la rappresentazione di -54 in base 10 in complemento a 2 su 8 bit.

- (A) 1101.1100
- (B) 1100 1010
- (C) 1101 1010
- (D) 0011 0110
- (E) Nessuna delle risposte

Risposta corretta: (B)

Perché è corretta: Si parte dal numero positivo 54_{10} , che in binario su 8 bit è 0011 0110 ($32 + 16 + 4 + 2$). Per ottenere il complemento a 2, si fa prima il complemento a 1 (si invertono tutti i bit: 1100 1001) e poi si somma 1. Aggiungendo 1, l'ultimo bit diventa 0 con riporto, ottenendo 1100 1010.

Perché scartare le altre: La (A) è formattata come numero a virgola fissa. La (C) presenta un errore di calcolo nel bit di peso 16. La (D) è la rappresentazione del numero positivo +54.

14. **Domanda:** Seleziona la rappresentazione binaria del numero 173.

- (A) 1010 1101
- (B) 1011 1101
- (C) 1010 1001
- (D) 1110 1101
- (E) Nessuna delle risposte

Risposta corretta: (A)

Perché è corretta: Il metodo più veloce è la somma dei pesi (potenze di 2): $173 = 128 + 32 + 8 + 4 + 1$. Mettendo a 1 le posizioni corrispondenti a questi pesi e a 0 le altre (64, 16, 2), si ottiene esattamente la sequenza 1010 1101.

Perché scartare le altre: La (B) vale 189 (ha il bit del 16 acceso). La (C) vale 169 (manca il bit del 4). La (D) vale 237 (ha il bit del 64 acceso).

15. **Domanda:** Convertire il numero 13.625 in base 10 in binario con rappresentazione a virgola fissa.

- (A) 1101.1100
- (B) 1110.1010
- (C) 1101.1010
- (D) 1101.0101

(E) Nessuna delle risposte

Risposta corretta: (C)

Perché è corretta: Si scompone in parte intera e frazionaria. La parte intera 13 è 1101 ($8 + 4 + 1$). La parte frazionaria 0.625 può essere vista come $\frac{1}{2} + \frac{1}{8}$, ovvero $0.5 + 0.125$. Nelle potenze di 2 post-virgola ($2^{-1}, 2^{-2}, 2^{-3}$), questo si traduce in 1 sul peso del mezzo, 0 sul quarto, e 1 sull'ottavo: .101. Unendo e aggiungendo uno zero di *padding* a destra (che non altera il valore) otteniamo 1101.1010.

Perché scartare le altre: La (A) equivale a 13.75 ($\frac{1}{2} + \frac{1}{4}$). La (B) calcola male la parte intera (14). La (D) equivale a 13.3125 ($\frac{1}{4} + \frac{1}{16}$).

16. **Domanda:** Quale di queste affermazioni è corretta nella comparazione tra architetture CISC e RISC?

- (A) Entrambe consentono di avere operandi sia in memoria che nei registri.
- (B) Entrambe codificano le istruzioni con un numero di bit costante.
- (C) L'architettura RISC non supporta operandi "immediati", mentre CISC li supporta.
- (D) L'architettura RISC non supporta l'esecuzione di istruzioni diverse in base al valore di verità di una condizione.
- (E) Entrambe dispongono di operazioni che consentono di scambiare dati tra la memoria e i registri.

Risposta corretta: (E)

Perché è corretta: Che un'architettura sia basata su istruzioni semplici (RISC) o complesse (CISC), lo scambio di dati tra la memoria centrale e la CPU è un'esigenza fondamentale. Entrambe possiedono operazioni dedicate a questo scopo (come `load/store` nel RISC-V o `MOV` nell'x86).

Perché scartare le altre: La (A) è falsa perché il RISC forza l'uso esclusivo di registri per le operazioni ALU, vietando operandi in memoria. La (B) è falsa perché il CISC (come l'Intel x86) usa istruzioni a lunghezza variabile. La (C) è falsa (il RISC usa moltissimo gli immediati, es. `addi`). La (D) è falsa (il RISC ha eccome i salti condizionati, es. `beq`).

17. **Domanda:** Il registro x5 contiene 11. Quale sarà il valore di x6 dopo `slli x6, x5, 2` seguito da `xori x6, x6, 15`?

- (A) x6 = 0000 0000 0000 0000 0000 0000 0010 0011
- (B) x6 = 0000 0000 0000 0000 0000 0000 0010 1100
- (C) x6 = 0000 0000 0000 0000 0000 0000 0011 0011
- (D) x6 = 0000 0000 0000 0000 0000 0000 0010 0111
- (E) Nessuna delle risposte

Risposta corretta: (A)

Perché è corretta: Il valore decimale 11 è 0000 1011. L'istruzione `slli` lo trasla a sinistra di 2 bit, diventando 0010 1100 (equivalente a moltiplicare per 4, cioè 44). L'istruzione `xori` fa un XOR bit a bit con 15 (0000 1111). La regola dell'XOR dice che l'uscita è 1 se e solo se i bit in ingresso sono diversi. Operando 0010 1100 XOR

0000 1111, otteniamo esattamente 0010 0011 (il valore 35).

Perché scartare le altre: La (B) ignora completamente il passaggio finale dell'XOR, fermandosi al primo shift. La (C) è il risultato di un'operazione OR invece di XOR. La (D) è il risultato di una banale addizione (+15) invece dell'XOR logico.

18. **Domanda:** Quale istruzione va inserita al posto di X1 nel ciclo `while(arr[i] != 0)`?

- (A) `bne a4, zero, .L3`
- (B) `beq a4, zero, .L2`
- (C) `bne a0, zero, .L3`
- (D) `beq a5, zero, .L2`
- (E) Nessuna delle risposte

Risposta corretta: (A)

Perché è corretta: X1 si trova alla fine del corpo del ciclo, fungendo da condizione per saltare indietro (.L3) e ripetere l'iterazione. L'elemento corrente dell'array è appena stato caricato nel registro a4. Dato che il C richiede di ciclare finché l'elemento è diverso da zero, l'istruzione deve ordinare "salta a .L3 se a4 NON È UGUALE a zero", ovvero `bne a4, zero, .L3`.

Perché scartare le altre: La (B) salta alla fine del ciclo se il valore è zero; sebbene logicamente corretto per uscire, posizionata in X1 richiederebbe poi un salto incondizionato esplicito per tornare su, allungando il codice inutilmente. La (C) controlla il contatore a0 invece dell'elemento letto dalla memoria. La (D) controlla l'indirizzo del puntatore a5, non il dato che esso contiene.

19. **Domanda:** Effettua la seguente sottrazione in binario: 1010 1100 - 0101 0111.

- (A) 0101 0101
- (B) 0101 1101
- (C) 0110 0101
- (D) 0100 0101
- (E) Nessuna delle risposte

Risposta corretta: (A)

Perché è corretta: Eseguendo la sottrazione colonna per colonna da destra verso sinistra e gestendo i prestiti (borrow), si ottiene 0101 0101. Un metodo di riprova infallibile è convertire i numeri in base 10: $172 - 87 = 85$. Convertendo 85 in binario ($64 + 16 + 4 + 1$) otteniamo proprio 0101 0101.

Perché scartare le altre: Le altre opzioni presentano semplici errori aritmetici nella propagazione dei prestiti tra i bit adiacenti.

20. **Domanda:** Rappresenta il numero decimale -5.75 nella codifica dello standard IEEE-754 a singola precisione (32 bit).

- (A) 0100 0000 1011 1000 0000 0000 0000 0000
- (B) 1100 0000 1101 1000 0000 0000 0000 0000
- (C) 1100 0000 1011 1000 0000 0000 0000 0000

(D) 1100 0001 0011 1000 0000 0000 0000 0000

(E) Nessuna delle risposte

Risposta corretta: (C)

Perché è corretta: 1) Segno: negativo $\rightarrow$ 1. 2) Conversione di 5.75 in binario: 101.11. 3) Normalizzazione: spostiamo la virgola di 2 posizioni $\rightarrow 1.0111 \times 2^2$. 4) Esponente con Bias: l'esponente base è 2, sum bias (127) = 129. 129 in binario è 1000 0001. 5) Mantissa: prendiamo 0111 (rimuovendo l'1 implicito) e riempiamo con zeri fino a 23 bit. Mettendo in fila i campi [1] [10000001] [01110000...] e raggruppando a blocchi di 4, otteniamo esattamente l'opzione C.

Perché scartare le altre: La (A) imposta il bit di segno a 0 (come fosse positivo). La (B) riporta una mantissa sbagliata (presume una frazione diversa da .75). La (D) ha un esponente pari a 130 anziché 129 (corrisponderebbe al numero -11.5).

21. **Domanda:** Converti il numero decimale positivo 213 nella sua rappresentazione binaria.

(A) 1101 0101

(B) 1100 0101

(C) 1101 0011

(D) 1110 0101

(E) Nessuna delle risposte

Risposta corretta: (A)

Perché è corretta: Utilizzando il metodo delle potenze decrescenti di due: $213 - 128 = 85$. $85 - 64 = 21$. $21 - 16 = 5$. $5 - 4 = 1$. $1 - 1 = 0$. Le potenze utilizzate sono 128, 64, 16, 4, 1. Accendendo i bit corrispondenti (1) e tenendo spenti gli altri (0), la sequenza prodotta è 1101 0101.

Perché scartare le altre: Ciascuna opzione errata somma un totale differente: la (B) equivale a 197, la (C) a 211 e la (D) a 229.

22. **Domanda:** Quale istruzione va inserita al posto di X1 nel calcolo del $\max(a, b)$?

(A) blt a0, a1, .L2

(B) bgt a0, a1, .L2

(C) bge a0, a1, .L2

(D) bne a0, a1, .L2

(E) Nessuna delle risposte

Risposta corretta: (C)

Perché è corretta: La logica Assembly assume temporaneamente che a (in $a0$) sia il massimo copiandolo in $a5$. L'istruzione X1 decide se dobbiamo scavalcare (saltare) il ramo dell'*else* (che eleggerebbe b come massimo). Dato che il C dice `if (a > b)`, dobbiamo saltare l'*else* se e solo se la condizione $a \geq b$ è vera (che copre sia $a > b$ sia $a == b$, per cui è indifferente chi dei due restituiamo). Pertanto l'istruzione per "saltare a .L2 se $a0 \geq a1$ " è `bge a0, a1, .L2`.

Perché scartare le altre: La (A) salterebbe l'*else* se $a < b$, producendo il risultato opposto (la funzione restituirebbe il minimo). La (B) salta se $a > b$, ma fallirebbe

nel caso in cui siano uguali (eseguendo un passaggio inutile o creando potenziali inefficienze logiche). La (D) salta sulla disuguaglianza pura, un criterio non adatto a un operatore maggiore/minore.

23. **Domanda:** Quale istruzione va inserita al posto di X1 per avanzare di tre posizioni nel ciclo `somma_ogni_tre`?

- (A) `addi a5, a5, 3`
- (B) `slli a5, a5, 2`
- (C) `addi a5, a5, 12`
- (D) `addi a4, a4, 12`
- (E) Nessuna delle risposte

Risposta corretta: (C)

Perché è corretta: X1 si occupa di aggiornare il *puntatore* base dell'array (`a5`) in modo che alla successiva iterazione l'istruzione `lw` legga l'elemento tre celle più avanti. In C, gli array di `int` usano celle di 4 byte ciascuna. Avanzare di 3 posizioni significa spostarsi in avanti di $3 \times 4 = 12$ byte fisici in memoria. L'istruzione corretta somma quindi 12 all'indirizzo in `a5`.

Perché scartare le altre: La (A) somma 3 byte, disallineando la lettura e corrompendo l'accesso alla memoria. La (B) moltiplica per 4 l'intero indirizzo base fisico, distruggendolo del tutto. La (D) altera l'indice del ciclo (`a4`), che invece l'Assembly precedente aggiornava già correttamente di +3 dal punto di vista logico.

24. **Domanda:** Quale istruzione va inserita al posto di X1 per implementare il costrutto `if(arr[i] > 0)`?

- (A) `bge a4, zero, .L3`
- (B) `ble a4, zero, .L3`
- (C) `bgt a4, zero, .L3`
- (D) `blt a4, zero, .L3`
- (E) Nessuna delle risposte

Risposta corretta: (B)

Perché è corretta: Come di consueto in Assembly, per decidere se eseguire o meno un blocco `if`, si valuta la **condizione opposta** rispetto al codice C. L'obiettivo è saltare l'operazione di incremento del contatore (`count++`) e passare alla successiva iterazione (`j .L3`). La condizione opposta a "maggiore di zero" (`> 0`) è "minore o uguale a zero" (`<= 0`). L'istruzione per "salta se `a4` è minore o uguale a zero" è `ble a4, zero, .L3`.

Perché scartare le altre: La (A) e la (C) si basano sulla condizione diritta e non opposta, il che comporterebbe il salto del blocco proprio quando andrebbe eseguito. La (D) salta unicamente se il numero è strettamente negativo, fallendo però nel caso in cui il numero sia esattamente zero (che non va conteggiato e quindi richiederebbe di saltare il blocco).